

Secure UART: End-to-End Encrypted Communication

Digital System Interfacing
Course Code: DSI

Student Names & IDs:

Youssef Basem (240919)
Moaaz Tamer (245789)
Marina Ayman (246635)
Gamila Hasan (242127)
Ramy Emad (241897)

Instructor:
Gehad Mohey

May 13, 2026

Abstract

This project presents the design and implementation of a secure communication channel between two microcontrollers over a standard UART interface. Utilizing the DH-Lite key exchange protocol and the Speck 32/64 lightweight block cipher, the system ensures data confidentiality and integrity against passive eavesdropping. A third "attacker" node is integrated into the system to demonstrate the effectiveness of the encryption by sniffing the communication lines and displaying the inability to decipher the intercepted traffic.

Contents

1	Introduction	3
2	Problem Statement	3
3	Objective	3
4	System Architecture	3
4.1	State Machine Transitions	4
5	System Components	4
5.1	Hardware Components	4
5.2	Software Stack	4
6	Cryptographic Foundations	5
6.1	DH-Lite: 8-bit Diffie-Hellman	5
6.2	Speck 32/64: Lightweight Block Cipher	5
7	Implementation Details	5
8	Code Snippets	6
8.1	Speck Encryption Logic (Alice)	6
8.2	DH-Lite Modular Exponentiation	6
9	Results	6
10	Discussion	7
11	Future Improvements	7
12	Conclusion	7

1 Introduction

UART (Universal Asynchronous Receiver-Transmitter) is a widely used communication protocol in embedded systems due to its simplicity and low overhead. However, UART transmits data in plaintext by default, making it highly vulnerable to sniffing and unauthorized data access. As IoT devices become more prevalent, securing low-level communication channels is critical. This project addresses these security concerns by layering a lightweight cryptographic protocol over the standard UART link.

2 Problem Statement

In most embedded applications, sensor data and control commands are sent over serial lines without any encryption. An adversary with physical access to the communication wires can easily intercept this data using a logic analyzer or a secondary microcontroller. The challenge is to implement a robust encryption scheme that fits within the limited memory and processing power of 8-bit microcontrollers like the ATmega328P.

3 Objective

The primary objectives of this project are:

- To implement a secure handshake using DH-Lite for shared secret generation.
- To utilize the Speck 32/64 block cipher for efficient, high-speed data encryption.
- To build a multi-node system (Alice, Bob, and Attacker) to validate the security measures.
- To provide real-time visualization of the key exchange and decrypted messages on LCD displays.

4 System Architecture

The system architecture follows a "Man-in-the-Middle" observation model. The communication between Alice and Bob is intercepted by a Sniffer node that has physical access to the UART lines but lacks the cryptographic keys to decrypt the payload.

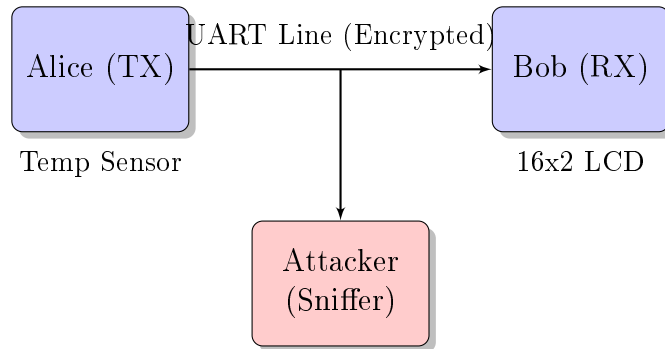


Figure 1: System block diagram showing the passive sniffing architecture.

4.1 State Machine Transitions

To ensure synchronization during the Diffie-Hellman handshake, both nodes implement a synchronous state machine.

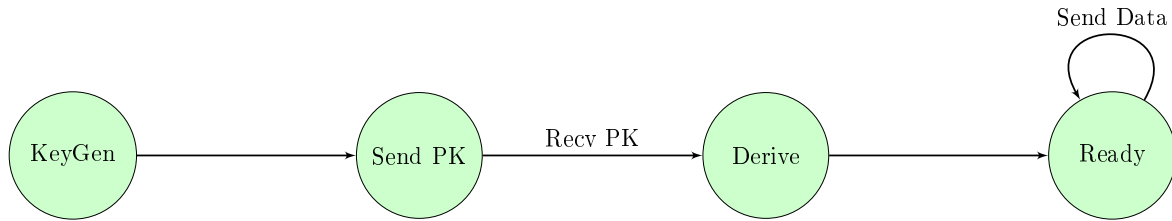


Figure 2: Handshake and data transmission state machine.

5 System Components

5.1 Hardware Components

- 3x Arduino Uno (ATmega328P).
- 2x 16x2 LCD Displays (for Bob and Attacker).
- LM35 Temperature Sensor.
- Push Button and Resistors.

5.2 Software Stack

- Arduino IDE / C++ for firmware.
- DH-Lite (Custom 8-bit implementation).
- Speck 64/128 (Lightweight block cipher).
- LiquidCrystal Library for LCD interfacing.

6 Cryptographic Foundations

The security of the system relies on two distinct cryptographic primitives: one for key agreement and another for symmetric encryption.

6.1 DH-Lite: 8-bit Diffie-Hellman

The Diffie-Hellman (DH) protocol allows two parties to establish a shared secret over an insecure channel. In this project, we implement a lightweight version called "DH-Lite" using:

- **Prime (p):** 251 (The largest 8-bit prime).
- **Generator (g):** 6 (A primitive root modulo 251).

The security is based on the *Discrete Logarithm Problem* (DLP). While an 8-bit prime is small for modern standards, it serves as an effective proof-of-concept for resource-constrained 8-bit microcontrollers where computing $g^a \pmod{p}$ is instantaneous but brute-forcing the private exponent a requires iterating through the entire group.

6.2 Speck 32/64: Lightweight Block Cipher

Speck is a family of lightweight block ciphers designed by the NSA for IoT devices. We utilize the Speck 32/64 variant:

- **Block Size:** 32 bits (Split into two 16-bit words x and y).
- **Key Size:** 64 bits (Four 16-bit words).
- **Round Function:** Uses ARX (Addition-Rotation-XOR) operations.
- **Number of Rounds:** 22.

Speck is chosen over AES due to its minimal RAM footprint and its optimization for 16-bit register architectures. In our implementation, using 16-bit words significantly reduces the instruction count compared to the 32-bit variant, making it ideal for the 8-bit ATmega328P.

7 Implementation Details

The firmware is written in C++ and optimized for the ATmega328P. Key implementation challenges included:

1. **Randomness:** The initial private keys are seeded using the `analogRead()` noise from an unconnected pin (A1) to ensure different shared secrets across sessions.
2. **Framing:** A framing protocol using start (0xAA) and end (0x55) bytes ensures that Bob can synchronize with the beginning of an encrypted block even if the serial line experiences noise.

3. **Floating Point Avoidance:** All temperature calculations for the LM35 and cryptographic operations use integer arithmetic to minimize binary size and execution time.
4. **Dual-Line Sniffing:** The sniffer node overcomes the single hardware UART limitation of the ATmega328P by utilizing *SoftwareSerial* on Pin 6 to monitor Bob's transmissions, while using the hardware RX pin to monitor Alice. This allows for a complete, bidirectional interception of the handshake.

8 Code Snippets

8.1 Speck Encryption Logic (Alice)

The following snippet demonstrates the 22-round Speck encryption core using 16-bit words and bitwise rotations.

```
1 void speck_encrypt(uint16_t* x, uint16_t* y) {
2   for (uint8_t i = 0; i < SPECK_ROUNDS; i++) {
3     *x = (ROR16(*x, 7) + *y) ^ roundKeys[i];
4     *y = ROL16(*y, 2) ^ *x;
5   }
6 }
```

8.2 DH-Lite Modular Exponentiation

The modular exponentiation function is optimized for 8-bit registers by using 16-bit intermediate results to prevent overflow during multiplication.

```
1 uint8_t dh_lite_modpow(uint8_t base, uint8_t exp, uint8_t mod) {
2   uint16_t result = 1;
3   uint16_t b = base % mod;
4   while (exp > 0) {
5     if (exp & 1) result = (result * b) % mod;
6     exp >>= 1;
7     b = (b * b) % mod;
8   }
9   return (uint8_t)result;
10 }
```

9 Results

The system successfully established a secure link. During testing:

- Alice and Bob correctly generated the same shared secret (e.g., 142).
- Temperature data (e.g., "TMP:25C") was correctly decrypted by Bob and displayed.
- The Attacker successfully intercepted the bytes but displayed them as unreadable hexadecimal values, confirming that the Speck cipher effectively obscured the payload.

10 Discussion

The choice of Speck 32/64 was optimal for this project because it operates natively on 16-bit words, which is significantly more efficient for the 8-bit ATmega328P compared to 32-bit or 64-bit ciphers. By reducing the number of rounds to 22 and using smaller data types, we achieved a high throughput while maintaining a minimal RAM footprint. The 8-bit DH-Lite provides a basic level of security suitable for educational purposes, though a larger prime would be required for production-level security. The dual-line sniffing proved that even with full visibility of the handshake, the discrete logarithm problem protects the shared secret.

11 Future Improvements

While the current implementation provides a robust proof-of-concept, several enhancements could be made for production environments:

- **Larger Key Space:** Upgrading DH-Lite to a 16-bit or 32-bit prime would significantly increase the complexity of brute-force attacks on the shared secret.
- **Message Integrity (MAC):** Integrating a Message Authentication Code (MAC) or an AEAD mode would ensure that data has not been tampered with during transmission, protecting against active "Man-in-the-Middle" attacks.
- **Hardware Entropy:** Replacing *analogRead()* with a dedicated hardware random number generator or a thermal noise circuit would provide more cryptographically secure private keys.
- **Key Rotation:** Implementing a session timer to automatically trigger a new DH handshake would ensure forward secrecy, limiting the impact of a potential key compromise.

12 Conclusion

This project demonstrated that end-to-end encryption is feasible on low-power microcontrollers. By combining a lightweight key exchange with a modern block cipher, we successfully secured a UART link against passive attacks. The modular design of the code allows for easy scaling to more complex sensor networks or different communication interfaces.